

TP N°8: Introducción a Ruby

La siguiente guía cubre los contenidos vistos en la clase teórica: **Introducción a Ruby**

Para verificar que se cuenta con Ruby instalado ejecutar desde la consola el comando

```
ruby --version
```

y verificar que cuenta con la versión 3.4 o superior

Para utilizar la consola de Ruby ejecutar desde la consola el comando `irb`. Para salir de la consola de Ruby invocar a `exit`.

Ejercicio 1

Retomando el *Ejercicio 1* del *TP N°1* se cuenta con el siguiente script de Ruby:

```
chainring = 52
cog        = 11
ratio      = chainring / cog.to_f
puts ratio # => 4.7272727272727275

chainring = 30
cog        = 27
ratio      = chainring / cog.to_f
puts ratio # => 1.1111111111111112
```

Se desea crear una clase **Gear** que ofrezca un comportamiento similar al del script. Diseñar el diagrama de clases correspondiente. Luego, en un archivo **gear.rb**, implementar la clase **Gear** y debajo de la definición escribir un programa de prueba para obtener los mismos valores que los del script pero utilizando la nueva clase **Gear**.

- Luego de haber ejecutado el programa, separar la definición de la clase **Gear** y el programa de prueba en dos archivos fuentes distintos **gear.rb** y **gear_tester.rb**. Ejecutar **gear_tester.rb**. ¿Cómo se indica en el archivo del programa de prueba donde se encuentra la definición de la clase **Gear**?
- Los ciclistas utilizan el término *gear inches* para comparar bicicletas de distinto *gearing* y tamaño de rueda. La fórmula utilizada es la siguiente:

$$\text{gear inches} = \text{gear ratio} * \text{wheel diameter}$$

donde

$$\text{wheel diameter} = \text{rim diameter} + \text{twice tire diameter}$$

Actualizar el diagrama de clases y modificar la clase **Gear** para que funcione con el siguiente programa de prueba:

```
require_relative 'gear'

rim = 26
tire = 1.5
puts Gear.new(52, 11, rim, tire).gear_inches # => 137.0909090909091
rim = 24
tire = 1.25
puts Gear.new(52, 11, rim, tire).gear_inches # => 125.27272727272728
```

C. Con el siguiente programa de prueba y la implementación anterior se obtiene el siguiente error:

```
require_relative 'gear'

puts Gear.new(52, 11).ratio # Esto funcionaba
...
wrong number of arguments (given 2, expected 4)
```

Como todos los parámetros son obligatorios, no se puede instanciar a la clase `Gear` sin especificar todos sus parámetros. Se busca poder seguir utilizando la clase `Gear` como en el apartado B, con sólo los primeros dos parámetros y así poder invocar el método `ratio`.

Para ello, crear una nueva clase `Wheel` que encapsule a las variables `rim` y `tire`. ¿Qué métodos deberían estar en esta clase? ¿Cómo cambia entonces la clase `Gear`? Realice una composición para que la clase `Gear` cuente con una instancia de la clase `Wheel`. Actualizar el diagrama de clases. Realizar la implementación de forma de poder ejecutar el siguiente programa de prueba:

```
require_relative 'gear'
require_relative 'wheel'

puts Gear.new(52, 11, Wheel.new(26, 1.5)).gear_inches # => 137.0909090909091
puts Gear.new(52, 11).ratio # => 4.7272727272727275
```

D. Realizar los cambios necesarios para que la clase `Gear` sea responsable de instanciar a la clase `Wheel`. Actualizar el diagrama de clases. De esta forma no se invoca al método constructor de la clase `Wheel` desde el programa de prueba:

```
require_relative 'gear'

puts Gear.new(52, 11, 26, 1.5).gear_inches # => 137.0909090909091
puts Gear.new(52, 11).ratio # => 4.7272727272727275
```

Ejercicio 2

Retomando el **Ejercicio 2** del **TP N°1**, implementar la clase `ComplexNumber` que modela un número complejo (parte real y parte imaginaria).

La clase debe permitir que dos instancias de números complejos puedan sumarse retornando una nueva instancia con el resultado de la operación.

Diseñar el diagrama de clases correspondiente e implementar todo lo necesario para que con el siguiente programa de prueba se obtenga la salida indicada en el comentario:

```
first_complex = ComplexNumber.new(2, -1)
second_complex = ComplexNumber.new(3, 0)
res_complex = first_complex + second_complex
puts res_complex.real.to_s + ' + ' + res_complex.imaginary.to_s + 'i'
# Imprime 5 + -1i
```

Luego de haber implementado la clase, respecto a la última línea del ejemplo:

- a) ¿Qué se obtiene si se quitan las invocaciones a `to_s` de `real` y `imaginary`?
- b) ¿Dónde corresponde que esté implementado la impresión de un número complejo?
- c) ¿Cómo se puede simplificar la última línea del ejemplo de invocación?

Ejercicio 3

Retomando el **Ejercicio 3** del **TP N°1**, implementar la clase `Point` que modela un punto en el plano 2D y ofrece un método que calcula la distancia entre dos puntos. El programa de prueba es el siguiente:

```
my_point = Point.new(1.5, 2.5)
puts my_point # Imprime {1.5, 2.5}
puts my_point.distance(Point.new(1.5, 3.0)) # Imprime 0.5
```

Ejercicio 4

Implementar la familia de clases de figuras geométricas del **Ejercicio 8** del **TP N°1** de tal forma que para el siguiente programa de prueba:

```
require_relative 'rectangle'
require_relative 'triangle'
require_relative 'ellipse'
require_relative 'circle'

start_point = Point.new(0, 0)

my_rectangle = Rectangle.new(start_point, Point.new(2, 4))
puts my_rectangle
puts my_rectangle.perimeter
puts my_rectangle.area

my_triangle = Triangle.new(start_point, Point.new(1, 0), Point.new(0, 1))
puts my_triangle
puts my_triangle.perimeter
puts my_triangle.area

my_ellipse = Ellipse.new(start_point, 2, 1)
puts my_ellipse
puts my_ellipse.perimeter
puts my_ellipse.area

my_circle = Circle.new(start_point, 3)
puts my_circle
puts my_circle.perimeter
```

```
puts my_circle.area
```

se obtenga la siguiente salida

```
Rectángulo [ {0, 0} , {2, 4} ]
12
8
Triángulo [ {0, 0} , {1, 0} , {0, 1} ]
3.414213562373095
0.49999999999999983
Elipse [Centro: {0, 0}, DMayor: 2, DMenor: 1]
4.71238898038469
1.5707963267948966
Círculo [Centro: {0, 0} , Radio: 3]
18.84955592153876
28.274333882308138
```

Ejercicio 5

Implementar la familia de clases de cuentas bancarias del *Ejercicio 10* del *TP N°1* de tal forma que para el siguiente programa de prueba:

```
require_relative 'checking_account'
require_relative 'savings_account'

my_checking_account = CheckingAccount.new(1001, 50)
my_checking_account.deposit(100.0)
puts my_checking_account
my_checking_account.extract(150.0)
puts my_checking_account

my_savings_account = SavingsAccount.new(1002)
my_savings_account.deposit(100.0)
puts my_savings_account
my_savings_account.extract(150.0)
```

se obtenga la siguiente salida

```
Cuenta 1001 con saldo 100.0
Cuenta 1001 con saldo -50.0
Cuenta 1002 con saldo 100.0
...
No cuenta con los fondos necesarios (RuntimeError)
```

Ejercicio 6

Retomando el ejercicio del *Taller* donde se modeló la familia de clases del **sistema de tarjetas de viaje en subte** se pide implementarlas de forma que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```
require_relative 'subway_central'
require_relative 'rechargeable_subway_card'
```

```

require_relative 'fixed_subway_card'
require_relative 'discount_rechargeable_subway_card'

subway_central = SubwayCentral.new(7.5)

rechargeable_card = RechargeableSubwayCard.new(subway_central)
rechargeable_card.recharge(15)
rechargeable_card.ride
puts rechargeable_card # Rechargeable Subway Card [Central: Subway Central [$7.5]]
                        [Balance: $7.5]
subway_central.ride_cost = 8.00
puts rechargeable_card # Rechargeable Subway Card [Central: Subway Central [$8.0]]
                        [Balance: $7.5]
# rechargeable_card.ride # Not enough funds (RuntimeError)

fixed_card = FixedSubwayCard.new(subway_central, 3)
fixed_card.ride
fixed_card.ride
puts fixed_card # Fixed Subway Card [Central: Subway Central [$8.0]] [Rides: 1]
fixed_card.ride
#fixed_card.ride # No rides available (RuntimeError)

discount_card = DiscountRechargeableSubwayCard.new(subway_central, 0.5)
discount_card.recharge(10)
discount_card.ride
puts discount_card # Rechargeable Subway Card [Central: Subway Central [$8.0]]
                  [Balance: $6.0]

```

Para las líneas de código comentadas, en caso de descomentarlas, el programa aborta con el mensaje indicado en el comentario.

Agregar luego las validaciones necesarias para los parámetros numéricos. Por ejemplo, para el siguiente fragmento de código:

```
subway_central = SubwayCentral.new('Hello World!')
```

el programa debería abortar con el siguiente mensaje:

```
Invalid Ride Cost (RuntimeError)
```

Ejercicio 7

Consultar la documentación de la clase `Object` e indicar en papel la salida de las siguientes expresiones. Luego comprobarlo con la consola.

```

a = String.new('Hola Mundo')
b = String.new('Hola Mundo')
puts a == b
puts a === b
puts a.eql? b
puts a.equal? b
puts a <=> b

```

Indicar el propósito de cada uno de los métodos. ¿Cuáles recomienda la documentación sobrescribir en las subclases?

Ejercicio 8

Indicar en papel la salida del siguiente programa de prueba que utiliza la clase `Point` implementada en el **Ejercicio 3**. Luego verificarlo en el entorno Ruby.

```
my_point = Point.new(1, 2)
puts my_point == my_point
puts my_point == Point.new(1, 2)
puts my_point != Point.new(3, 4)
puts my_point == 'Hola Mundo'
```

Agregar a la clase `Point` todo lo necesario para que la clase pueda responder correctamente a la **equivalencia**.

Ejercicio 9

Agregar a la familia de clases de figuras geométricas del **Ejercicio 4** todo lo necesario para que las clases puedan responder correctamente a la **equivalencia**. Actualice el diagrama de clases correspondiente. Con el siguiente programa de prueba:

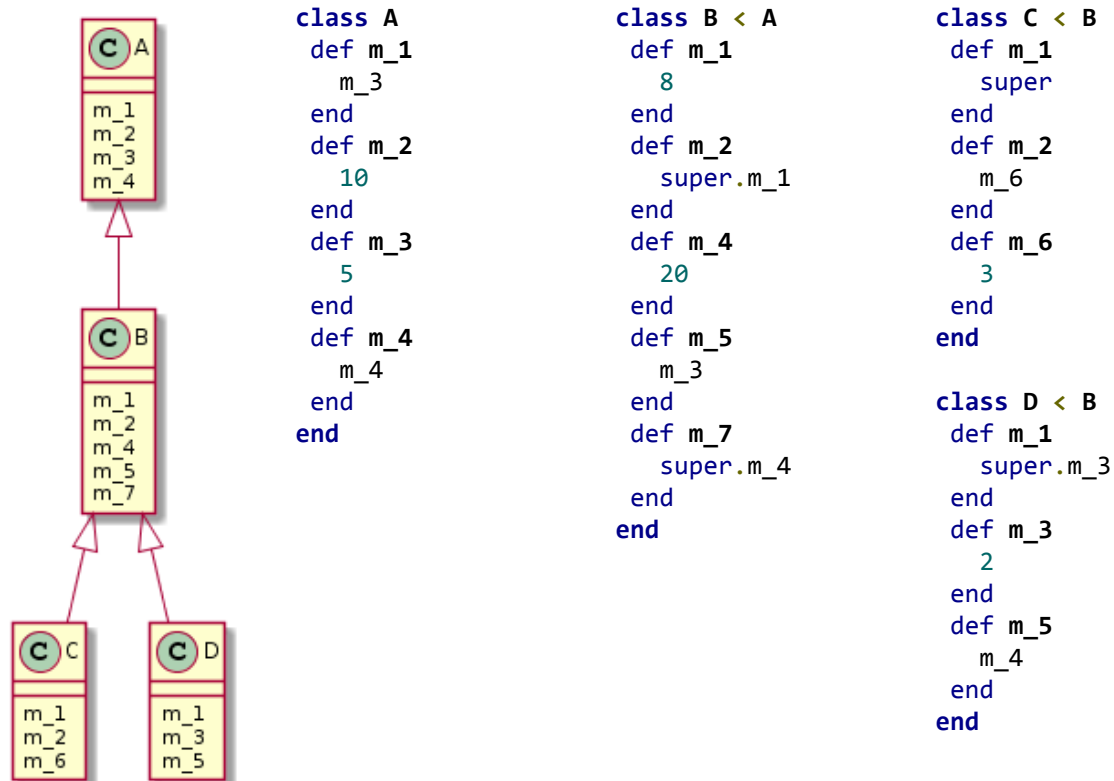
```
my_rectangle = Rectangle.new(Point.new(0, 0), Point.new(2, 4))
puts my_rectangle == Rectangle.new(Point.new(0, 0), Point.new(2, 4))
my_circle = Circle.new(Point.new(0, 0), 3)
puts my_circle == Circle.new(Point.new(0, 0), 3)
puts my_rectangle == my_circle
```

se debe obtener la siguiente salida:

```
true
true
false
```

Ejercicio 10

Dada la siguiente jerarquía de clases, con los métodos de instancia indicados para cada una, se cuenta con cuatro instancias homónimas a la clase a la cual pertenecen.



Completar el siguiente cuadro indicando qué se obtiene al enviar los mensajes de la primera fila (por separado, no en cascada) a la instancia de la primera columna. Primero hacerlo en papel para verificar que se entendió la invocación a métodos y el uso de la palabra reservada `super`. Luego verificarlo en el entorno Ruby.

	m_1	m_2	m_3	m_4	m_5	m_6	m_7
A							
B							
C							
D							